Concurrent Progress Bars (max 3 at once)

Problem: Clicking "Add" adds a progress bar. Each bar fills from 0 to 100% in a fixed time. At most 3 bars may fill at the same time; extra bars wait in a queue and start, in order, as soon as a slot frees up.
Tech stack: React (Vite, JavaScript) (default, none given)
Time limit: 60 minutes (default, none given)
Often asked at: Uber, Meta. **Why it matters:** queue plus concurrency limit.

PART A: Thought process

General steps for any machine coding task

1. **Clarify** the requirements and pin down scope. Write the assumptions down.
2. **List the interactions** (action → expected result). This is the build and test checklist.
3. **Sketch the components** and who owns which state.
4. **Design the state**: the smallest set of facts; derive everything else.
5. **Find the hard parts** early and decide how to solve them before coding.
6. **Build the simplest working version**, then add one working layer at a time.
7. **Cover the states and edge cases** (empty, running, paused, finished, reset).
8. **Wrap up**: trade-offs, what you would add with more time.

Time budget (60 minutes)

Minutes	Activity
0-7	Requirements, clarifying questions, assumptions
7-15	Use cases, components, state model, the "running = derived" idea
15-40	Build: one static bar → one animating bar → many bars → concurrency limit
40-50	Pause / resume, reset, counts, accessibility
50-55	Edge cases, quick manual test
55-60	Wrap-up and trade-offs

Think out loud

Say every decision and its reason as you make it: "I'll keep only progress in state, because running vs queued can be derived from order; storing it would mean two sources of truth." The interviewer grades the reasoning, not just the final UI. If you get stuck, say what you are trying and why.

PART B: Apply it to this problem

1. Understand the requirements

Functional requirements

- An **"Add bar"** button appends a new bar to the end of the list.
- Each bar fills from 0% to 100% in **2 seconds** (fixed).
- At most **3 bars fill at the same time**. Any bar beyond that is **queued** and waits.
- When a running bar finishes, the **oldest queued bar starts** (FIFO).
- Each bar shows its state: **queued**, **running (n%)**, or **done**.
- Extras (asked for as follow-ups, cheap to add): **Pause / Resume** all, **Reset**, a summary line "Running 3 · Queued 2 · Done 4".

Non-functional requirements

Area	What it means here
Performance	One shared timer for all bars, not one per bar. Animate with `transform`, not `width`. No timer runs when nothing is running.
Correctness	Never more than 3 running; strict FIFO; progress capped at 100; no leaked intervals on unmount.
Timing accuracy	Progress based on real elapsed time, so a slow or throttled timer does not stretch a 2 s bar to 5 s.
Error handling	No network here; the "errors" are timer leaks, stale state in closures, double ticks in StrictMode.
Accessibility	`role="progressbar"` with `aria-valuenow/min/max` and a label; a polite live region for the summary; real `<button>`s with focus styles.
Reusability	The queue logic is pure functions (`tick`, `getRunningIds`) that can be tested and reused with any limit or duration.
Extensibility	Limit and duration are constants/params; the same model works for real uploads where progress comes from events.

Clarifying questions to ask

1. Is the limit always 3, or should it be configurable? (Build with a constant, pass it as a param.)
2. Do all bars take the same time, or can each have its own duration? (Same, 2 s.)
3. Should queued bars start in the order they were added (FIFO)? (Yes.)
4. Do finished bars stay on screen or disappear? (Stay, marked done.)
5. Is there pause / cancel / reset? (Pause all and reset as extras; per-bar cancel is a "more time" item.)
6. Is the progress fake (timer) or from real work like uploads? (Fake timer; real work is a follow-up.)

Assumptions we build with

- Limit = **3**, duration = **2000 ms**, both constants.

- FIFO queue; bars are never reordered.
- Finished bars stay visible. Reset clears all bars.
- Pause freezes every running bar; resume continues from the same percentage.
- No persistence; refreshing the page starts over.

2. Identify interactions / use cases

Action / input	Expected behavior / output
Page loads	Empty message, "Running 0 · Queued 0 · Done 0", Pause and Reset disabled
Click "Add bar" once	Bar #1 appears and starts filling right away
Click "Add bar" 5 times quickly	Bars #1-#3 running, #4-#5 show "queued" with a striped track
A running bar reaches 100%	It turns green "done"; the oldest queued bar starts in the same tick
Add a bar while 3 are running	New bar appears as queued at the end
Add a bar after all are done	It starts immediately (a slot is free)
Click "Pause"	All running bars freeze; summary shows "Paused"; button reads "Resume"
Click "Resume"	Bars continue from where they stopped
Add bars while paused	They are added (queued or running slot reserved) but nothing moves until resume
Click "Reset"	All bars removed, pause cleared, timer stopped
Tab hidden for a while, then visible	Bars jump to the correct progress for the real time passed (no slow-motion)
Screen reader	Each bar is announced as a progressbar with its value and status; summary changes are announced politely

3. Design the architecture

Module	Responsibility
`utils/queue.js`	Pure logic: constants, `createBar`, `getRunningIds`, `getStatus`, `tick`, `getCounts`
`hooks/useProgressQueue.js`	Owns the bars and paused state, runs the single shared interval, exposes `add / togglePause / reset`
`components/ProgressBar.jsx`	Dumb view of one bar: track, fill, label, status, ARIA
`App.jsx`	Layout, buttons, summary line, maps bars to `ProgressBar`

```
App
 ├─ useProgressQueue()  ── uses ──> utils/queue.js (pure)
 │     state: bars[], paused
 │     effect: one setInterval while (running > 0 && !paused)
 ├─ controls: Add · Pause/Resume · Reset
 ├─ summary (aria-live)
 └─ <ul> ProgressBar × n   (status derived per render)
```

Start with everything in `App.jsx` (one bar, one interval). Split the pure logic out as soon as the "who is running?" rule appears, because that is the part you want to test and reuse.

4. Design the state / data model

State or field	Purpose
`bars: { id, progress }[]`	The queue in insertion order. `progress` is 0-100.
`paused: boolean`	Whether the shared timer is allowed to run
`last` (local to the effect)	Timestamp of the previous tick, for real elapsed time

```
const [bars, setBars] = useState([]);      // [{ id: 1, progress: 42.5 }, ...]
const [paused, setPaused] = useState(false);
```

Senior note: derive, don't store.

- `status` (queued / running / done) is **not** stored. Done = `progress >= 100`. Running = the **first 3 bars with progress < 100**. Queued = the rest. Because the list is in insertion order, "first 3 unfinished" *is* FIFO with a limit of 3.
- Storing `status` would need a second piece of code to "promote" the next queued bar when one finishes, and the two can drift apart (4 running, or a stuck queued bar). Derived status cannot be wrong.
- Counts (`running / queued / done`) are derived too.
- **Ownership:** the hook owns `bars` and `paused`. `ProgressBar` owns nothing; it gets `progress` and `status` as props.

5. Design the data structures and contracts

Example data

```
[
  { id: 1, progress: 100 },  // done
  { id: 2, progress: 64.2 }, // running
  { id: 3, progress: 64.2 }, // running
  { id: 4, progress: 12.0 }, // running (slot freed by #1)
  { id: 5, progress: 0 },    // queued
]
```

Stable, increasing `id`s (a module counter) are used as React keys, so bars keep their DOM nodes when the list grows or is reset. Using the array index as key would work here because we only append, but ids are the safe default and are needed for per-bar cancel later.

Contracts (pure functions, no API)

Function	Input	Output
`createBar()`	none	`{ id, progress: 0 }`
`getRunningIds(bars, limit = 3)`	bars	`Set` of ids of the first `limit` unfinished bars
`getStatus(bar, runningIds)`	bar, set	`'done' | 'running' | 'queued'`
`tick(bars, dt, limit = 3, duration = 2000)`	bars, elapsed ms	new bars array; running ones advanced by `dt / duration * 100`, capped at 100. Returns the **same array** when nothing runs.
`getCounts(bars, limit = 3)`	bars	`{ running, queued, done }`

Hook contract: `useProgressQueue()` → `{ bars, counts, paused, add, togglePause, reset }`.

6. Trace the data flow

Key interaction: bar #1 finishes while #4 is queued.

```
setInterval fires (every 50 ms)
   │  dt = now - last  (e.g. 50 ms)
   ▼
setBars(prev => tick(prev, dt))
   │  running = first 3 unfinished = {1,2,3}
   │  #1: 98.0 + 2.5 → min(100, 100.5) = 100
   ▼
React re-renders App
   │  getRunningIds(bars) = first 3 unfinished = {2,3,4}   ← #4 promoted, no extra code
   │  getCounts → Running 3 · Queued 1 · Done 1
   ▼
ProgressBar #1 → status "done" (green)
ProgressBar #4 → status "running"
   │
   ▼
next tick advances {2,3,4}
...
last bar hits 100 → running = 0 → hasWork false
   ▼
effect cleanup → clearInterval  (no idle timer)
```

7. Important technical challenges

Challenge 1: Enforcing "max 3" and FIFO without a scheduler

Problem: The obvious approach is a `running` counter plus a `queue` array: on add, if `running < 3` start it, else push to queue; on finish, decrement and shift the queue. With async timers this is easy to break: two bars finish in the same tick, the counter is decremented twice but only one item is shifted, or a closure reads an old counter and starts a 4th bar.

Solution: Store only progress. Running is *derived* as the first `LIMIT` unfinished bars. The rule is one line, cannot drift, and promotion happens automatically on the next render.

```
export const getRunningIds = (bars, limit = LIMIT) =>
  new Set(bars.filter((b) => b.progress < 100).slice(0, limit).map((b) => b.id));
```

Challenge 2: One timer, not N timers, and no leaks

Problem: One `setInterval` per bar means 100 bars → 100 timers, each calling `setState`, and each one must be cleared on finish, pause, reset and unmount. Missing one leaks a timer that keeps updating state.

Solution: A single interval in the hook, driven by two booleans. It exists only while there is running work and we are not paused; React's effect cleanup clears it in every case (pause, finish, reset, unmount). The updater form `setBars(prev => ...)` means the interval never reads stale `bars`.

```
useEffect(() => {
  if (!hasWork || paused) return;
  let last = performance.now();
  const id = setInterval(() => {
    const now = performance.now();
    const dt = now - last;
    last = now;
    setBars((prev) => tick(prev, dt));
  }, TICK_MS);
  return () => clearInterval(id);
}, [hasWork, paused]);
```

Note that adding bars while 3 are running does **not** restart the interval: `hasWork` stays `true`, so the effect does not re-run.

Challenge 3: Accurate timing (throttled timers, background tabs)

Problem: If each tick adds a fixed `+2.5%` (assuming exactly 50 ms), any delay slows the bar down. Browsers throttle timers in background tabs to about once per second, so a 2 s bar could take 40 s.

Solution: Advance by the **real elapsed time** (`dt` from `performance.now()`), so a late tick simply moves the bar further. `tick` is pure and takes `dt`, which also makes it trivial to test.

```
const step = (dt / duration) * 100;
return bars.map((b) =>
  running.has(b.id) ? { ...b, progress: Math.min(100, b.progress + step) } : b,
);
```

`last` is reset when the effect starts, so time spent paused is never counted.

Challenge 4: Smooth rendering

Problem: Setting `width: 42%` on every tick triggers layout for every bar 20 times per second.

Solution: Animate the fill with `transform: scaleX(p)` and `transform-origin: left`; transforms are composited and skip layout. `tick` also returns the same array when nothing is running, so React can bail out of a re-render.

```
<div className="bar__fill" style={{ transform: `scaleX(${progress / 100})` }} />
```

8. Implement in the correct order

#	Step	Working result
1	Static `ProgressBar` with a hard-coded 40%	One bar renders correctly

#	Step	Working result
2	`bars` state + "Add bar" button	Bars appear at 0%
3	One `setInterval` advancing **all** bars	Everything fills (no limit yet)
4	Add `getRunningIds` and only advance those	Max 3 fill, rest wait, FIFO promotion works
5	Show derived status (queued / running % / done)	Visual states visible
6	Stop the interval when nothing runs (`hasWork` dep)	No idle timer
7	Switch to real elapsed time (`dt`)	Accurate under throttling
8	Move logic to `utils/queue.js` + `useProgressQueue`	Testable pure logic
9	Pause / Resume, Reset, summary counts	Controls done
10	ARIA (`progressbar` , live region), disabled buttons, `scaleX`	Accessible and smooth
11	Run through the use-case table; assert-test `tick`	Verified

9. Cover all states and edge cases

State / edge case	Behavior
Initial / empty	"No bars yet" message; Pause and Reset disabled; no interval running
Running (≤ 3 bars)	All bars fill at once
Over the limit	First 3 run, the rest show "queued" with a striped track
One bar finishes	Marked done; next queued bar starts on the same render
Several finish in the same tick	All freed slots filled at once (derived rule, no counter bugs)
All done	Interval cleared; Pause disabled; "Done n"
Add after all done	New bar starts immediately; interval restarts
Paused	Bars frozen; adding bars still works; Resume continues exactly where it stopped
Paused time	Not counted, because `last` is reset when the effect restarts
Reset while running / paused	All bars cleared, `paused = false` , interval cleared by cleanup
Spam-clicking Add (hundreds)	One timer total; only 3 bars update; list grows linearly
Tick overshoot (`dt` larger than remaining)	Clamped to 100 with `Math.min`
Background tab	Bars catch up to real time when the tab is visible again
Unmount	Effect cleanup clears the interval
React StrictMode double effects	Effect runs, cleans up, runs again: still exactly one interval

10. Final structure

```
_66_concurrent_progress_bars/
├─ index.html
├─ package.json
├─ vite.config.js
└─ src/
   ├─ main.jsx
   ├─ App.jsx                  # layout, controls, summary, list
   ├─ index.css
   ├─ components/
   │  └─ ProgressBar.jsx       # one bar, ARIA, scaleX fill
   ├─ hooks/
   │  └─ useProgressQueue.js   # bars + paused state, single shared interval
   └─ utils/
      └─ queue.js              # pure queue logic: tick, getRunningIds, getCounts
```

11. Full working code

`src/utils/queue.js`

```
export const LIMIT = 3; // max bars filling at the same time
export const DURATION = 2000; // ms for one bar to go 0 -> 100

let nextId = 1;
export const createBar = () => ({ id: nextId++, progress: 0 });

// Running bars are never stored: they are the first LIMIT unfinished bars (FIFO).
export const getRunningIds = (bars, limit = LIMIT) =>
  new Set(bars.filter((b) => b.progress < 100).slice(0, limit).map((b) => b.id));

export const getStatus = (bar, runningIds) =>
  bar.progress >= 100 ? 'done' : runningIds.has(bar.id) ? 'running' : 'queued';

// Pure: advance only the running bars by the real time that passed (dt ms).
export function tick(bars, dt, limit = LIMIT, duration = DURATION) {
  const running = getRunningIds(bars, limit);
  if (running.size === 0) return bars;
  const step = (dt / duration) * 100;
  return bars.map((b) =>
    running.has(b.id) ? { ...b, progress: Math.min(100, b.progress + step) } : b,
  );
}

export function getCounts(bars, limit = LIMIT) {
  const running = getRunningIds(bars, limit).size;
  const done = bars.filter((b) => b.progress >= 100).length;
  return { running, done, queued: bars.length - running - done };
}
```

`src/hooks/useProgressQueue.js`

```
import { useEffect, useState } from 'react';
import { createBar, getCounts, tick } from '../utils/queue';

const TICK_MS = 50;

export function useProgressQueue() {
  const [bars, setBars] = useState([]);
  const [paused, setPaused] = useState(false);

  const counts = getCounts(bars);
  const hasWork = counts.running > 0;

  // One shared timer for all bars; it only exists while there is work and we are not paused.
  useEffect(() => {
    if (!hasWork || paused) return;
    let last = performance.now();
    const id = setInterval(() => {
      const now = performance.now();
      const dt = now - last; // real elapsed time, so throttled timers don't slow the bars
      last = now;
      setBars((prev) => tick(prev, dt));
    }, TICK_MS);
    return () => clearInterval(id);
  }, [hasWork, paused]);

  return {
    bars,
    counts,
    paused,
    add: () => setBars((prev) => [...prev, createBar()]),
    togglePause: () => setPaused((p) => !p),
    reset: () => {
      setBars([]);
      setPaused(false);
    },
  };
}
```

`src/components/ProgressBar.jsx`

```
export default function ProgressBar({ index, progress, status }) {
  const value = Math.floor(progress);
  return (
    <li className={`bar bar--${status}`}>
      <span className="bar__label">#{index}</span>
      <div
        className="bar__track"
        role="progressbar"
        aria-label={`Bar ${index}, ${status}`}
        aria-valuemin={0}
        aria-valuemax={100}
        aria-valuenow={value}
      >
        {/* scaleX instead of width: animates on the compositor, no layout per tick */}
        <div className="bar__fill" style={{ transform: `scaleX(${progress / 100})` }} />
      </div>
      <span className="bar__status">{status === 'running' ? `${value}%` : status}</span>
    </li>
  );
}
```

`src/App.jsx`

```
import ProgressBar from './components/ProgressBar';
import { useProgressQueue } from './hooks/useProgressQueue';
import { getRunningIds, getStatus, LIMIT } from './utils/queue';

export default function App() {
  const { bars, counts, paused, add, togglePause, reset } = useProgressQueue();
  const runningIds = getRunningIds(bars);

  return (
    <main className="app">
      <h1>Concurrent progress bars</h1>
      <p className="hint">At most {LIMIT} bars fill at once; the rest wait in a queue.</p>

      <div className="controls">
        <button onClick={add}>Add bar</button>
        <button onClick={togglePause} disabled={counts.running === 0}>
          {paused ? 'Resume' : 'Pause'}
        </button>
        <button onClick={reset} disabled={bars.length === 0}>
          Reset
        </button>
      </div>

      <p className="counts" aria-live="polite">
        Running {counts.running} · Queued {counts.queued} · Done {counts.done}
        {paused && ' · Paused'}
      </p>

      {bars.length === 0 ? (
        <p className="empty">No bars yet. Click “Add bar”.</p>
      ) : (
        <ul className="bars">
          {bars.map((bar, i) => (
            <ProgressBar
              key={bar.id}
              index={i + 1}
              progress={bar.progress}
              status={getStatus(bar, runningIds)}
            />
          ))}
        </ul>
      )}
    </main>
```

```
  );
}
```

src/index.css

```
* { box-sizing: border-box; }
body { margin: 0; font-family: system-ui, sans-serif; background: #f6f7f9; color: #1f2328; }

.app { max-width: 560px; margin: 40px auto; padding: 0 16px; }
.hint, .empty { color: #57606a; }

.controls { display: flex; gap: 8px; }
button { padding: 8px 14px; border: 1px solid #d0d7de; border-radius: 6px; background: #fff; cursor:
pointer; font: inherit; }
button:hover:not(:disabled) { background: #f3f4f6; }
button:disabled { opacity: 0.5; cursor: not-allowed; }
button:focus-visible { outline: 2px solid #0969da; outline-offset: 2px; }

.counts { font-variant-numeric: tabular-nums; }

.bars { list-style: none; padding: 0; display: grid; gap: 10px; }
.bar { display: grid; grid-template-columns: 36px 1fr 64px; align-items: center; gap: 10px; }
.bar__label, .bar__status { font-size: 14px; font-variant-numeric: tabular-nums; }
.bar__status { text-align: right; color: #57606a; }
.bar__track { height: 14px; background: #e5e7eb; border-radius: 7px; overflow: hidden; }
.bar__fill { height: 100%; background: #0969da; transform-origin: left; }
.bar--done .bar__fill { background: #1a7f37; }
.bar--queued .bar__track { background: repeating-linear-gradient(45deg, #e5e7eb 0 6px, #eef0f2 6px
12px); }
```

src/main.jsx

```
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import './index.css'
import App from './App.jsx'

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>,
)
```

Quick self-check of the pure logic (`node check.mjs`)

```
import assert from 'node:assert/strict';
import { createBar, getCounts, getRunningIds, tick } from './src/utils/queue.js';

let bars = Array.from({ length: 5 }, createBar);
assert.deepEqual(getCounts(bars), { running: 3, queued: 2, done: 0 });
bars = tick(bars, 1000);                        // half way
assert.deepEqual(bars.map((b) => b.progress), [50, 50, 50, 0, 0]);
bars = tick(bars, 5000);                        // overshoot clamps, slots free up
assert.deepEqual(getCounts(bars), { running: 2, queued: 0, done: 3 });
assert.deepEqual([...getRunningIds(bars)], [bars[3].id, bars[4].id]);
bars = tick(bars, 2000);                        // remaining two finish
assert.deepEqual(getCounts(bars), { running: 0, queued: 0, done: 5 });
assert.equal(tick(bars, 50), bars);             // nothing to do -> same reference
console.log('self-check ok');
```

12. Wrap-up

Edge cases handled

- Never more than 3 running, strict FIFO, multiple finishes in one tick.
- Progress capped at 100; time while paused is not counted.
- Accurate timing under throttled timers and background tabs.
- One shared interval that only exists while needed; cleared on pause, finish, reset and unmount.
- Empty, paused and all-done states with correctly disabled buttons.
- Accessible progress bars and a polite live summary.

Trade-offs to mention

- **Derived running set vs explicit queue:** derived is simpler and cannot drift, but it re-scans the list each tick (O(n)). Fine for hundreds of bars; for 100k you would keep an index of the first unfinished bar.
- **setInterval vs requestAnimationFrame:** rAF is smoother and syncs with paint, but it stops completely in background tabs. With `dt` the interval catches up correctly; switching to rAF is a one-line change in the effect.
- **Leftover time is dropped:** if a bar needs 10 ms to finish and the tick was 50 ms, the next bar does not get the other 40 ms. Invisible at 50 ms ticks; exact carry-over would complicate `tick` for no visible gain.
- **One state update per tick for all bars** re-renders the list; `React.memo` on `ProgressBar` would skip done/queued bars if lists get large.

What to add with more time

- Per-bar cancel / retry, and a configurable limit (input or prop).

- Different duration per bar (store `duration` on the bar; `tick` uses it).
- Real work instead of fake timers: a promise pool with the same limit (see Q15).
- Unit tests with Vitest + fake timers; React Testing Library tests for the UI flow.
- Virtualize the list and memoize rows for thousands of bars.

13. Counter questions the interviewer may ask

Requirements

Q1. Why did you ask whether the limit is configurable before writing code?

Because it decides whether "3" is hard-coded inside logic or passed in. I made it a constant and also a parameter of the pure functions, so changing it to a prop or an input later touches one line.

Q2. Should a finished bar free its slot immediately or after an animation?

Immediately: the requirement is about concurrent *work*. The derived rule frees the slot on the same render the bar reaches 100. A "done" animation is pure CSS on the `.bar--done` class and does not hold a slot.

State design

Q3. Why not store a `status` field on each bar?

It duplicates information that order + progress already give. Stored status needs promotion code on every finish, and any bug there leaves 4 running or a bar stuck in "queued". Derived status is always consistent.

Q4. Why not keep a `running` counter and a separate `queue` array?

Two pieces of state that must change together in async callbacks. Two bars finishing in the same tick, or a stale closure, can make them disagree. One list with a derived rule removes the whole class of bug.

Q5. Why is `last` a local variable inside the effect and not a `useRef`?

It only matters for the lifetime of one interval. A fresh `last` each time the effect starts also means paused time is automatically excluded. A ref would have to be reset manually on resume.

Challenge: concurrency limit

Q6. What happens if two bars finish in the same tick?

Both reach 100 in the same `tick`. On the next render the first-3-unfinished rule picks two new bars at once. No counter is involved, so nothing is double-decremented.

Q7. How would you allow a different limit per "lane" or priority?

Keep `priority` on each bar and sort the unfinished bars by `(priority, id)` before `slice(0, limit)`. For lanes, group by lane and apply the limit per group. The rule stays a pure function.

Challenge: timer

Q8. Why one interval instead of one per bar?

One timer means one place to start and stop, one `setState` per tick for all bars, and no per-bar cleanup. N timers scale badly and each one is a possible leak.

Q9. What does StrictMode do to your interval?

In development it mounts, unmounts and mounts again, so the effect runs, cleans up, and runs again. Because cleanup calls `clearInterval`, there is still exactly one interval. Without cleanup you would see bars filling at double speed.

Q10. Why does adding a bar not restart the interval?

The effect depends only on `hasWork` and `paused`. Adding a bar while others run keeps `hasWork` true, so the effect is not re-run and the timing is not disturbed.

Q11. Why not `requestAnimationFrame`?

rAF is smoother but pauses entirely in background tabs, and the requirement is "fill in 2 s", not "animate at 60 fps". With `dt` either works; I'd switch to rAF if smoothness matters more, and keep `dt` so it still catches up.

Implementation steps

Q12. Why is `tick` a pure function that takes `dt`?

It can be tested without timers or React: `tick(bars, 1000)` should be 50%. It also returns the same array when nothing runs, so React skips the update.

Q13. Why `transform: scaleX` instead of `width`?

`width` changes cause layout on every tick for every bar. `transform` is handled by the compositor and skips layout and paint of siblings, which matters with many bars updating 20 times per second.

Performance and "what if"

Q14. What if the user adds 10,000 bars?

Still one timer, and only 3 bars change per tick. The costs are the O(n) scan in `getRunningIds` and rendering 10,000 rows. Fixes: keep a pointer to the first unfinished index, `React.memo` the rows, and virtualize the list (only render visible rows).

Q15. What if each bar represented a real upload/API call instead of a timer?

Same queue idea, but the limit is applied to *starting promises*: a pool that runs at most 3 tasks and starts the next when one settles. Progress comes from `onprogress` events instead of `dt`.

```
async function runPool(tasks, limit = 3) {
  const results = []; let i = 0;
  const worker = async () => { while (i < tasks.length) { const k = i++; results[k] = await tasks[k]();
} };
  await Promise.all(Array.from({ length: Math.min(limit, tasks.length) }, worker));
  return results;
}
```

Q16. What if a real task fails?

Mark the bar `error` (store `error: true`, or a status only for the failure case), treat it as finished so its slot frees, and show a Retry button that resets its progress and moves it to the end of the queue (or keeps its place, depending on the product).

Q17. What if the tab is in the background for a minute?

Timers are throttled to about once per second, but `dt` uses real time, so on the next tick the bars jump forward the right amount. All bars may be done when you come back, which is correct.

Trade-offs and "why not X"

Q18. Why not `useReducer`?

There are only three actions (add, tick, reset) and they are already pure functions. A reducer is a good move once actions grow (cancel, retry, reorder, per-bar pause); the functions in `queue.js` would become its cases unchanged.

Q19. Why not CSS animations (`transition: transform 2s`) and no timer at all?

CSS alone cannot tell JavaScript when a bar is done in a reliable, testable way (you would depend on `transitionend`), pause/resume becomes `animation-play-state` juggling, and progress percentage is not readable for ARIA. The state-driven approach keeps one source of truth.

Q20. How would you test this?

Unit-test `tick`, `getRunningIds` and `getCounts` with plain asserts (done above). For the hook/UI, use Vitest fake timers and React Testing Library: add 5 bars, advance 2 s, expect 3 done and 2 running; pause, advance, expect no change.